

LAPORAN 3

JavaScript (Functions & Array)

Disusun untuk memenuhi tugas laporan



DISUSUN OLEH :

AMARA IMANIKA
NIM 2025573010042

Kelas : TI 1C
Mata Kuliah : Algoritma dan Struktur Data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025 - 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat-Nya, laporan praktikum mengenai "Struktur Data dan Algoritma: Function & Array" ini dapat diselesaikan dengan tepat waktu. Laporan ini disusun sebagai bentuk dokumentasi hasil pembelajaran pada Modul 3 yang berfokus pada pengembangan logika pemrograman menggunakan JavaScript.

Laporan ini mencakup pemahaman mendalam mengenai penggunaan fungsi, metode manipulasi array, hingga konsep tingkat lanjut seperti rekursi. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami dasar-dasar pengembangan aplikasi web yang efisien. Kritik dan saran sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Lhokseumawe, 13 April 2026

Penulis

Amara Imanika

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan Penulisan	4
1.3 Manfaat Penulisan	4
BAB II PEMBAHASAN	5
2.1 Alur kerja dan latihan Pemograman JavaScript (Functions & Array)	5
2.2 Tugas Pratikum	21
BAB III PENUTUP	26
3.1 Kesimpulan	26
3.2 Saran	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, khususnya pengembangan aplikasi menggunakan JavaScript dan Node.js, pengelolaan kode yang efisien sangatlah penting. Dua konsep utama yang mendasari efisiensi tersebut adalah *Function* (fungsi) dan *Array* (larik). Penggunaan fungsi memungkinkan pengembang untuk menerapkan prinsip DRY (*Don't Repeat Yourself*), di mana kode dapat digunakan kembali tanpa penulisan ulang. Di sisi lain, *Array* memungkinkan penyimpanan dan manipulasi kumpulan data secara terstruktur. Pemahaman atas kedua konsep ini, termasuk teknik modern seperti *Arrow Function* dan *Array Methods*, menjadi fondasi krusial bagi setiap pengembang perangkat lunak.

1.2 Tujuan Penulisan

Tujuan dari penulisan laporan ini adalah agar pembaca mampu:

1. Mendefinisikan dan memanggil fungsi dengan parameter serta nilai kembalian (*return value*).
2. Memahami perbedaan antara fungsi biasa dengan *Arrow Function* yang lebih modern.
3. Menguasai konsep *Callback* sebagai argumen dalam fungsi.
4. Manipulasi data dalam *Array* menggunakan metode bawaan seperti `push`, `pop`, `shift`, dan `unshift`.
5. Menerapkan metode iterasi modern seperti `forEach`, `map`, `filter`, dan `reduce` untuk pengolahan data.
6. Memahami teknik rekursi dalam memecahkan masalah komputasi secara bertahap.

1.3 Manfaat Penulisan

- **Bagi Penulis:** Memperkuat pemahaman logis mengenai struktur data dan algoritma dalam JavaScript serta memenuhi tugas praktikum Modul 3.
- **Bagi Pembaca:** Memberikan panduan praktis dan referensi mengenai implementasi fungsi dan array dalam skenario pemrograman nyata, seperti pembuatan sistem nilai mahasiswa atau kalkulator sederhana.

BAB II

PEMBAHASAN

2.1 Alur kerja dan latihan Pemograman JavaScript (Functions & Array)

Langkah 1 | Buka Git Bash. Kemudian navigation ke root folder repository kamu.

```
cd nama – repository - kamu
```

Langkah 2 | Buat folder dengan nama praktikum-3

```
mkdir praktikum-3  
cd praktikum-3
```

Langkah 3 | Buat folder dengan nama praktikum-3

```
npm init -y
```

Langkah 4 |

Buat file baru di dalam folder praktikum-3 dengan nama 01-function-dasar.js

Ketikkan kode 01-function-dasar.js dibawah ini :

```
function sapa() {  
  console.log('Halo! Selamat datang di pratikum JavaScript.');}  
sapa();  
sapa();
```

```
function sapaNama(nama) {  
  console.log(`Halo, #{nama}! Selamat belajar.`);  
}  
sapaNama('Budi');  
sapaNama('Siti');
```

```
function tambah(angka1, angka2) {  
  const hasil = angka1 + angka2;  
  return hasil;  
}
```

```
const hasilPenjumlahan = tambah(10,25);  
console.log('10 + 25 =', hasilPenjumlahan);  
console.log('7 + 13 =', tambah(7, 13));
```

```
function hitung(nilai, pengali = 2) {  
  return nilai * pengali;  
}
```

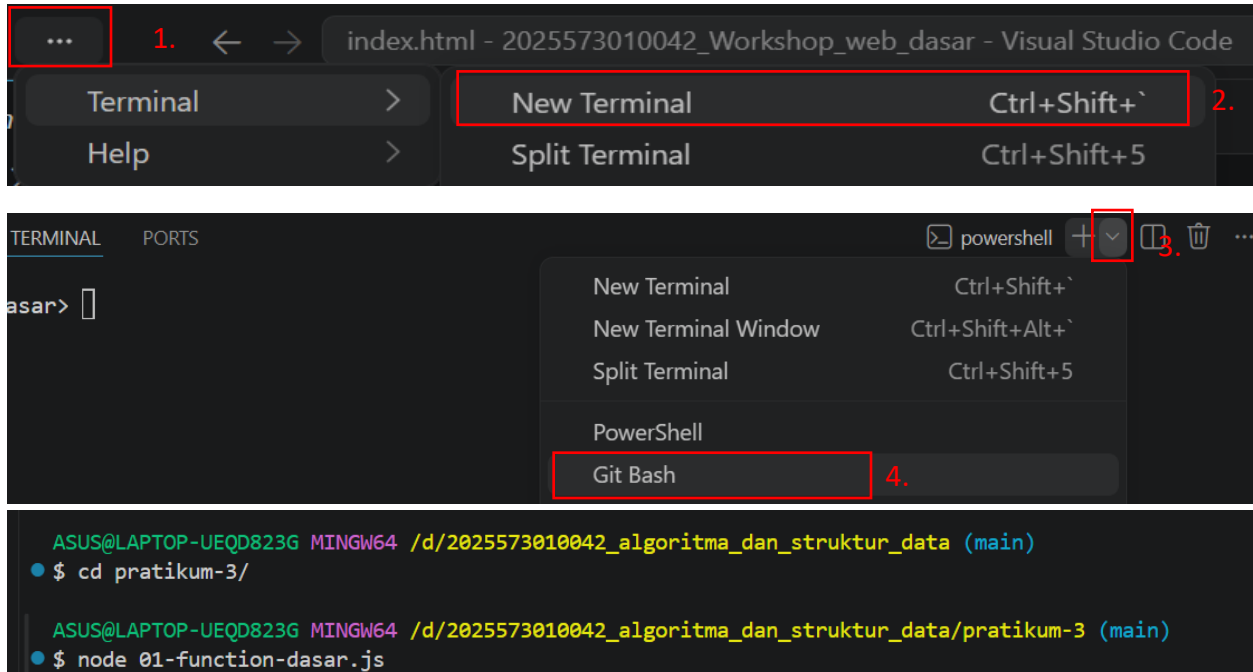
```
console.log(hitung(5));  
console.log(hitung(5, 3));
```

```
const pesanGlobal = 'Saya ada di mana saja';
```

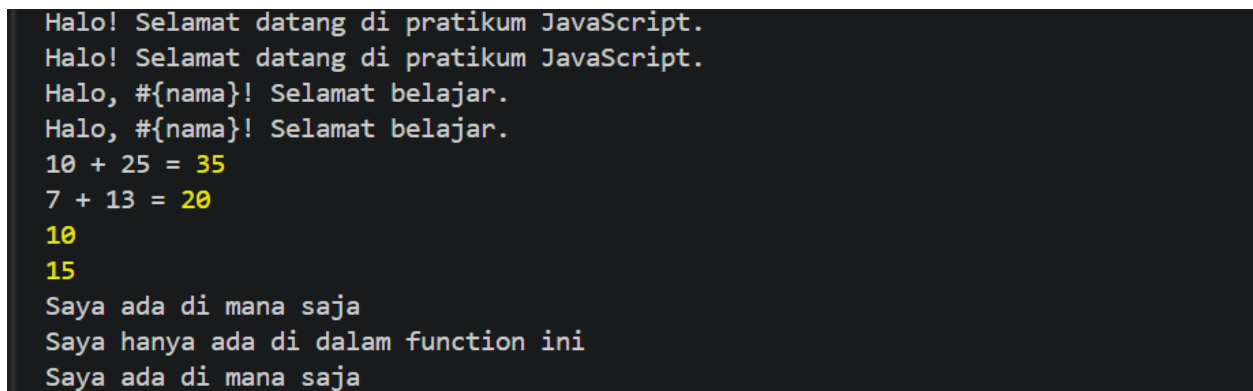
```
function cekScope() {  
  const pesanLokal = 'Saya hanya ada di dalam function ini';  
  console.log(pesanGlobal);  
  console.log(pesanLokal);  
}
```

```
cekScope();  
console.log(pesanGlobal);
```

Untuk menjalankan hasil di terminal, bisa mengikuti langkah dibawah ini. Simpan file (Ctrl + S atau Cmd + S), kemudian jalankan file tersebut dari terminal. Lebih jelasnya lihat gambar dibawah.



Hasilnya akan seperti gambar dibawah ini :



Ketikkan kode latihan 1 di bawah kode function dasar .

```
console.log('');
console.log('=== Latihan1 ===');
```

```
function kurang(a, b) {
  return a - b;
}
```

```
function kali(a, b) {
  return a * b;
}
```

```
function bagi(a, b) {
  if (b === 0) {
    console.log('Error: tidak bisa membagi dengan nol');
    return null;
  }
  return a / b;
}
```

```
function tambah(a, b) {
  return a + b;
}
```



```
function kalkulator(a, operator, b) {
  if (operator === '+') {
    return tambah(a, b);
  } else if (operator === '-') {
    return kurang(a, b);
  } else if (operator === '*') {
    return kali(a, b);
  } else if (operator === '/') {
    return bagi(a, b);
  } else {
    return "Operator tidak dikenal";
  }
}
```

```
console.log("Hasil Penjumlahan (10 + 5):", kalkulator(10, '+', 5));
console.log("Hasil Pengurangan (10 - 5):", kalkulator(10, '-', 5));
console.log("Hasil Perkalian (10 * 5):", kalkulator(10, '*', 5));
console.log("Hasil Pembagian (10 / 2):", kalkulator(10, '/', 2));
console.log("Hasil Pembagian Nol (10 / 0):", kalkulator(10, '/', 0));
```

Hasilnya akan seperti gambar dibawah ini :

```
=== Latihan1 ===
Hasil Penjumlahan (10 + 5): 15
Hasil Pengurangan (10 - 5): 5
Hasil Perkalian (10 * 5): 50
Hasil Pembagian (10 / 2): 5
Error: tidak bisa membagi dengan nol
Hasil Pembagian Nol (10 / 0): null
```

Langkah 5 | Buat file baru dengan nama 02-arrow-callback.js

Ketikkan kode 02-arrow-callback dibawah ini :

```
function kuadratBiasa(x) {  
  return x * x;  
}
```

```
const kuadrat = (x) => {  
  return x * x;  
};
```

```
const kuadratRingkas = x => x * x;
```

```
console.log('=== Perbandingan Penulisan ===');  
console.log('Biasa  :', kuadratBiasa(5));  
console.log('Arrow  :', kuadrat(5));  
console.log('Ringkas:', kuadratRingkas(5));
```

```
const luas = (panjang, lebar) => panjang * lebar;  
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}`;
```

```
console.log('Luas 4x6 :', luas(4, 6));  
console.log(salam('Budi', 'Pagi'));
```

```
function lakukanOperasi(angka, operasiCallback) {  
  console.log(`Angka awal: ${angka}`);  
  const hasil = operasiCallback(angka);  
  console.log(`Hasil setelah operasi: ${hasil}`);  
}
```

```
console.log('\n=== callback ===');  
lakukanOperasi(7, kuadratRingkas);  
lakukanOperasi(10, x => x + 100);  
lakukanOperasi(20, function(x) {  
  return x / 2;  
}));
```

```

console.log('\n=== setTimeout (Callback) ===');
console.log('Pesan 1: Sebelum timer');

setTimeout(() => {
  console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');
}, 100);

console.log('Pesan 2: Setelah mendaftarkan timer');

```

Hasilnya akan seperti gambar dibawah ini :

```

• $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-
• $ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa   : 25
Arrow   : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi

=== callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer

```

Latihan 2 | Pipeline Transformasi Data

```

console.log('');
console.log('=== Latihan2 ===');

const keHurufBesar = (str) => str.toUpperCase();

const tambahSeru = (str) => str + "!!!";

const hitungKata = (str) => str.split(' ').length;

```

```
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);

  console.log("Hasil transformasi:", hasil);
}

const teks = 'belajar javascript itu menyenangkan';

console.log("--- Menguji keHurufBesar ---");
prosesKalimat(teks, keHurufBesar);

console.log("\n--- Menguji tambahSeru ---");
prosesKalimat(teks, tambahSeru);

console.log("\n--- Menguji hitungKata ---");
prosesKalimat(teks, hitungKata);
```

Hasilnya akan seperti gambar dibawah ini :

```
=== Latihan2 ===
--- Menguji keHurufBesar ---
Hasil transformasi: BELAJAR JAVASCRIPT ITU MENYENANGKAN

--- Menguji tambahSeru ---
Hasil transformasi: belajar javascript itu menyenangkan!!!

--- Menguji hitungKata ---
Hasil transformasi: 4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Langkah 6 | Buat file baru dengan nama 03-array-dasar.js

```
const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
const nilai = [85, 92, 78, 88];

console.log('=== Array Awal ===');
console.log('Mahasiswa:', mahasiswa);
console.log('Nilai :', nilai);
console.log('Jumlah mahasiswa:', mahasiswa.length);
```

```

console.log('\n=== Akses Elemen ===');
console.log('Elemen pertama :', mahasiswa[0]);
console.log('Elemen ketiga :', mahasiswa[2]);
console.log('Elemen terakhir:', mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = 'Siti Rahayu';
console.log('\nSetelah diubah:', mahasiswa);

console.log('\n=== Manipulasi Array ===');
mahasiswa.push('Doni');
console.log('Setelah push :', mahasiswa);

mahasiswa.unshift('Zahra');
console.log('Setelah unshift :', mahasiswa);

const dihapusAkhir = mahasiswa.pop();
console.log('Dihapus (pop) :', dihapusAkhir);
console.log('Setelah pop :', mahasiswa);

const dihapusAwal = mahasiswa.shift();
console.log('Dihapus (shift) :', dihapusAwal);
console.log('Setelah shift :', mahasiswa);

console.log('\n=== Pencarian ===');
console.log('Indeks Ahmad :', mahasiswa.indexOf('Ahmad'));
console.log('Ada Rina? :', mahasiswa.includes('Rina'));
console.log('Ada Budi? :', mahasiswa.includes('Budi'));

const sebagian = nilai.slice(1, 4);
console.log('\nNilai asli :', nilai);
console.log('Slice [1,4] :', sebagian);

```

Hasilnya akan seperti gambar dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
● $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-3 (main)
● $ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 88 ]
Jumlah mahasiswa: 4

```

```
=== Akses Elemen ===  
Elemen pertama : Budi  
Elemen ketiga : Ahmad  
Elemen terakhir: Rina
```

```
Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
```

```
=== Manipulasi Array ===
```

```
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]  
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]  
Dihapus (pop) : Doni  
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]  
Dihapus (shift) : Zahra  
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
```

```
=== Pencarian ===
```

```
Indeks Ahmad : 2  
Ada Rina? : true  
Ada Budi? : true
```

```
Nilai asli : [ 85, 92, 78, 88 ]  
Slice [1,4] : [ 92, 78, 88 ]
```

Latihan 3 | Manajemen Daftar Belanja

```
console.log('');  
console.log('=== Latihan3 ===');  
  
let daftarBelanja = ['Beras', 'Gula', 'Telur', 'Minyak', 'Garam'];  
  
console.log("--- Daftar Belanja Awal ---");  
for (let i = 0; i < daftarBelanja.length; i++) {  
  console.log(`${i + 1}. ${daftarBelanja[i]}`);  
}  
  
daftarBelanja.push('Susu', 'Kopi');  
console.log("\n(Susu dan Kopi ditambahkan ke daftar)");  
  
let itemDihapus = daftarBelanja.shift();  
console.log(`Item yang dihapus dari daftar pertama: ${itemDihapus}`);
```

```

let adaSusu = daftarBelanja.includes('Susu');
if (adaSusu) {
  console.log("Cek Susu: Ya, Susu ada di dalam daftar belanja.");
} else {
  console.log("Cek Susu: Tidak, Susu tidak ada di daftar.");
}

console.log(`Jumlah total item sekarang: ${daftarBelanja.length} item.`);

console.log("Daftar akhir:", daftarBelanja);

```

Hasilnya akan seperti gambar dibawah ini :

```

=== Latihan3 ===
--- Daftar Belanja Awal ---
1. Beras
2. Gula
3. Telur
4. Minyak
5. Garam

(Susu dan Kopi ditambahkan ke daftar)
Item yang dihapus dari daftar pertama: Beras
Cek Susu: Ya, Susu ada di dalam daftar belanja.
Jumlah total item sekarang: 6 item.
Daftar akhir: [ 'Gula', 'Telur', 'Minyak', 'Garam', 'Susu', 'Kopi' ]

```

Langkah 7 | Buat file baru dengan nama 04-array-method.js

```

const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

console.log('=== forEach: Tampilkan Semua Nilai ===');
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

const gradeHuruf = nilaiMahasiswa.map(nilai => {
  if (nilai >= 90) return 'A';
  if (nilai >= 80) return 'B';
  if (nilai >= 70) return 'C';
  if (nilai >= 60) return 'D';
  return 'E';
});

```

```

console.log('\n=== map: Nilai ke Grade ===');
console.log('Nilai :', nilaiMahasiswa);
console.log('Grade :', gradeHuruf);

const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);

console.log('\n=== filter: Lulus dan Tidak Lulus ===');
console.log('Semua nilai :', nilaiMahasiswa);
console.log('Semua lulus :', nilaiLulus);
console.log('Semua gagal :', nilaiGagal);

const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0);

const rataRata = totalNilai / nilaiMahasiswa.length;

console.log('\n=== reduce: Statistik Nilai ===');
console.log('Total nilai :', totalNilai);
console.log('Rata-rata :', rataRata.toFixed(2));

const rataRataNilaiLulus = nilaiMahasiswa
  .filter(nilai => nilai >= 60)
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
  }, 0);

console.log('\n=== Method Chaining ===');
console.log('Rata-rata nilai lulus:', rataRataNilaiLulus.toFixed(2));

```


Hasilnya akan seperti gambar dibawah ini :

```
ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
● $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-3 (main)
● $ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
Mahasiswa-1: 75
Mahasiswa-2: 90
Mahasiswa-3: 55
Mahasiswa-4: 82
Mahasiswa-5: 68
Mahasiswa-6: 95
Mahasiswa-7: 48
Mahasiswa-8: 77
```

```
=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'E', 'A', 'E',
  'E', 'E', 'A',
  'E', 'E'
]
```

```
=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Semua lulus : [ 75, 90, 82, 68, 95, 77 ]
Semua gagal : [ 55, 48 ]
```

```
=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75
```

```
=== Method Chaining ===
Rata-rata nilai lulus: 81.17
```

```
console.log('');
console.log('=== Latihan4 ===');

const produk = [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Mouse', harga: 150000, stok: 0 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Monitor', harga: 3200000, stok: 0 },
  { nama: 'Headset', harga: 350000, stok: 8 }
];

const produkTersedia = produk.filter(p => p.stok > 0);
console.log("--- Produk Tersedia ---");
console.log(produkTersedia);

const namaProdukTersedia = produkTersedia.map(p => p.nama);
console.log("\n--- Nama Produk Tersedia ---");
console.log(namaProdukTersedia);

const totalInventaris = produkTersedia.reduce((total, p) => {
  return total + (p.harga * p.stok);
}, 0);
console.log("\nTotal Nilai Inventaris: Rp" + totalInventaris.toLocaleString('id-ID'));

console.log("\n--- Laporan Stok Produk ---");
produk.forEach(p => {
  const status = p.stok > 0 ? `[TERSEDIA] ${p.nama} - Rp${p.harga.toLocaleString('id-ID')} (${p.stok} unit)`
    : `[HABIS] ${p.nama} - Rp${p.harga.toLocaleString('id-ID')}`;
  console.log(status);
});
```

Hasilnya akan seperti gambar dibawah ini :

```
=== Latihan4 ===
--- Produk Tersedia ---
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]
```

```
--- Nama Produk Tersedia ---  
[ 'Laptop', 'Keyboard', 'Headset' ]
```

```
Total Nilai Inventaris: Rp50.700.000
```

```
--- Laporan Stok Produk ---  
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)  
[HABIS] Mouse - Rp150.000  
[TERSEDIA] Keyboard - Rp450.000 (12 unit)  
[HABIS] Monitor - Rp3.200.000  
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

Langkah 8 | Buat file baru dengan nama 05-rekursi.js

```
function faktorial(n) {  
    // Base case: berhenti saat n =1  
    if (n <= 1) return 1;  
  
    // Recursive case: n x faktorial(n-1)  
    return n * faktorial(n - 1);  
}
```

```
console.log('=== Faktorial ===');  
console.log('0! =', faktorial(0));  
console.log('1! =', faktorial(1));  
console.log('5! =', faktorial(5));  
console.log('7! =', faktorial(7));
```

```
function fibonacci(n) {  
    // Base case  
    if (n === 0) return 0;  
    if (n === 1) return 1;  
  
    //Recursive case  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```

console.log('\n=== Fibonacci ===');
for (let i = 0; i <= 8; i++) {
    process.stdout.write(fibonacci(i) + ' ');
}
console.log(''); //pindah baris

function jumlahArray(arr, indeks = 0) {
    //Base case: sudah sampai akhir array
    if (indeks === arr.length) return 0;

    //Recursive case: elemen saat ini + jumlah sisa array
    return arr[indeks] + jumlahArray(arr, indeks + 1);
}

const angka = [3, 7, 2, 9, 5];
console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array :', angka);
console.log('Jumlah :', jumlahArray(angka));

function countdown(n) {
    if (n < 0) {
        console.log('Selesai!');
        return;
    }

    console.log(n);
    countdown(n - 1);
}

console.log('\n=== Countdown Rekursi ===');
countdown(5);

```

Hasilnya akan seperti gambar dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-3 (main)
• $ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

```

```
=== Fibonacci ===
```

```
01123581321
```

```
=== Jumlah Array dengan Rekursi ===
```

```
Array : [ 3, 7, 2, 9, 5 ]
```

```
Jumlah : 26
```

```
=== Countdown Rekursi ===
```

```
5
```

```
4
```

```
3
```

```
2
```

```
1
```

```
0
```

```
Selesai!
```

2.2 Tugas Pratikum

Tugas	Buat folder baru dengan nama tugas.
-------	-------------------------------------

Setelah membuat folder tugas buat 2 file di dalamnya dengan nama tugas-1.js dan tugas-2.js. kemudian ketikkan kode dibawah ini :

Tugas 1	Sistem Nilai Mahasiswa
---------	------------------------

```
const dataMahasiswa = [
  { nama: "Desty", nilai: 85 },
  { nama: "Siti", nilai: 92 },
  { nama: "Fika", nilai: 58 },
  { nama: "Dilla", nilai: 70 },
  { nama: "Budi", nilai: 45 },
  { nama: "Bilqis", nilai: 80 }
];
```

```
function hitungStatistik(arrMahasiswa) {
  const hasilReduce = arrMahasiswa.reduce((acc, mhs) => {
    return {
      total: acc.total + 1,
      jumlahNilai: acc.jumlahNilai + mhs.nilai,
      tertua: Math.max(acc.tertua, mhs.nilai),
      terendah: Math.min(acc.terendah, mhs.nilai)
    };
  }, { total: 0, jumlahNilai: 0, tertua: -Infinity, terendah: Infinity });
```

```

    return {
      total: hasilReduce.total,
      rataRata: (hasilReduce.jumlahNilai / hasilReduce.total).toFixed(2),
      tertinggi: hasilReduce.tertua,
      terendah: hasilReduce.terendah
    };
  }
}

```

```

function filterLulus(arrMahasiswa, batasLulus) {
  return arrMahasiswa.filter(mhs => mhs.nilai >= batasLulus);
}

```

```

function tambahGrade(arrMahasiswa) {
  return arrMahasiswa.map(mhs => {
    let grade;
    if (mhs.nilai >= 85) grade = 'A';
    else if (mhs.nilai >= 75) grade = 'B';
    else if (mhs.nilai >= 65) grade = 'C';
    else if (mhs.nilai >= 50) grade = 'D';
    else grade = 'E';

    return { ...mhs, grade: grade };
  });
}

```

```

console.log("=== LAPORAN NILAI MAHASISWA ===");

```

```

const stats = hitungStatistik(dataMahasiswa);

```

```

console.log(`
--- Statistik Grup ---
Total Mahasiswa : ${stats.total}
Rata-rata Nilai : ${stats.rataRata}
Nilai Tertinggi : ${stats.tertinggi}
Nilai Terendah : ${stats.terendah}
`);

```

```

const daftarLulus = filterLulus(dataMahasiswa, 65);
console.log("--- Mahasiswa yang Lulus (Batas 65) ---");
daftarLulus.forEach(mhs => {
  console.log(`> ${mhs.nama} dinyatakan LULUS dengan nilai ${mhs.nilai}`);
});

```

```

const dataFinal = tambahGrade(dataMahasiswa);
console.log("\n--- Daftar Lengkap dengan Grade ---");
dataFinal.forEach((mhs, index) => {
  console.log(`${index + 1}. Nama: ${mhs.nama.padEnd(8)} | Nilai: ${mhs.nilai} | Grade: ${mhs.grade}`);
});

```

Hasilnya akan seperti gambar dibawah ini :

```
ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
● $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-3 (main)
● $ node tugas/tugas-1.js
=== LAPORAN NILAI MAHASISWA ===

--- Statistik Grup ---
Total Mahasiswa : 6
Rata-rata Nilai : 71.67
Nilai Tertinggi : 92
Nilai Terendah : 45

--- Mahasiswa yang Lulus (Batas 65) ---
> Desty dinyatakan LULUS dengan nilai 85
> Siti dinyatakan LULUS dengan nilai 92
> Dilla dinyatakan LULUS dengan nilai 70
> Bilqis dinyatakan LULUS dengan nilai 80

--- Daftar Lengkap dengan Grade ---
1. Nama: Desty      | Nilai: 85 | Grade: A
2. Nama: Siti      | Nilai: 92 | Grade: A
3. Nama: Fika      | Nilai: 58 | Grade: D
4. Nama: Dilla     | Nilai: 70 | Grade: C
5. Nama: Budi      | Nilai: 45 | Grade: E
6. Nama: Bilqis    | Nilai: 80 | Grade: B
```

Tugas 2 | Pangkat dan Palindrom Rekursif

```
function pangkat(basis, eksponen) {

    if (eksponen === 0) {
        return 1;
    }

    return basis * pangkat(basis, eksponen - 1);
}
```

```
function balikString(str) {  
  if (str.length <= 1) {  
    return str;  
  }  
  
  let karakterTerakhir = str[str.length - 1];  
  let sisaString = str.slice(0, str.length - 1);  
  
  return karakterTerakhir + balikString(sisaString);  
}
```

```
function cekPalindrom(str) {  
  
  const strKecil = str.toLowerCase();  
  const hasilBalik = balikString(strKecil);  
  
  return strKecil === hasilBalik;  
}
```

```
console.log("\n--- Uji Pangkat ---");  
console.log(`2 pangkat 3 = ${pangkat(2, 3)}`); // 8  
console.log(`5 pangkat 2 = ${pangkat(5, 2)}`); // 25
```

```
console.log("\n--- Uji Pangkat ---");  
console.log(`2 pangkat 3 = ${pangkat(2, 3)}`);  
console.log(`5 pangkat 2 = ${pangkat(5, 2)}`);
```

```
console.log("\n--- Uji Balik String ---");  
console.log(`'halo' dibalik menjadi: '${balikString('halo')}'`);  
console.log(`'javascript' dibalik menjadi: '${balikString('javascript')}'`);
```


Hasilnya akan seperti gambar dibawah ini :

```
ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-3/

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-3 (main)
• $ node tugas/tugas-2.js

--- Uji Pangkat ---
2 pangkat 3 = 8
5 pangkat 2 = 25

--- Uji Balik String ---
'halo' dibalik menjadi: 'olah'
'javascript' dibalik menjadi: 'tpircsavaj'

--- Uji Palindrom ---
Apakah 'katak' palindrom? YA
Apakah 'civic' palindrom? YA
Apakah 'level' palindrom? YA
Apakah 'belajar' palindrom? BUKAN
```

Langkah 9 | Submit ke GitHub

Setelah semua file selesai dibuat maka langkah terakhir yaitu melakukan push semua file ke GitHub. Untuk itu buka terminal kemudian pilih GitBash lalu ketikkan kode dibawah ini :

```
cd pratikum-3/

git add .

git commit -m "Pratikum 3: Function & Array – arrow function, callback, iterasi array, rekursi"

git push origin main
```

Apabila berhasil maka pada GitHub kamu akan muncul seperti tampilan pada gambar di bawah ini :

 amaraimanikaa	Pratikum 3: Function & Array - arrow function, callback, ...	d1ad320 · 4 days ago	
 pratikum-2	Upload ulang seluruh isi pratikum-2 (terma...	5 days ago	
 pratikum-3	Pratikum 3: Function & Array - arrow funct...	4 days ago	

Nah pratikum-3 yang barusan kita buat sudah muncul. Itu artinya push yang kita lakukan berhasil.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan materi pada Modul 3, dapat disimpulkan bahwa:

- **Function** merupakan blok kode yang mempermudah reusabilitas dan pengelolaan lingkup (*scope*) variabel.
- **Array Methods** seperti `map` dan `filter` sangat powerful untuk mentransformasi data tanpa mengubah array aslinya (*immutable*).
- **Rekursi** adalah teknik alternatif selain perulangan (*looping*) yang sangat berguna untuk menyelesaikan struktur data yang bersifat hierarkis atau berulang, seperti faktorial dan deret Fibonacci.
- Penggunaan **Asynchronous** melalui *callback* (seperti pada `setTimeout`) menunjukkan karakteristik utama JavaScript yang tidak menunggu satu proses selesai sebelum menjalankan perintah berikutnya.

3.2 Saran

- **Optimasi Rekursi:** Disarankan bagi praktikan untuk mempelajari teknik *memoization* atau *tail recursion* guna menghindari error *Maximum call stack size exceeded* pada kasus rekursi yang dalam.
- **Eksplorasi Metode Chaining:** Penggunaan *method chaining* (menggabungkan `filter` dan `reduce`) harus lebih sering dilatih untuk menghasilkan kode yang lebih ringkas dan mudah dibaca.
- **Penggunaan Konstanta:** Sebaiknya selalu menggunakan `const` untuk mendeklarasikan array guna mencegah perubahan referensi variabel yang tidak disengaja.